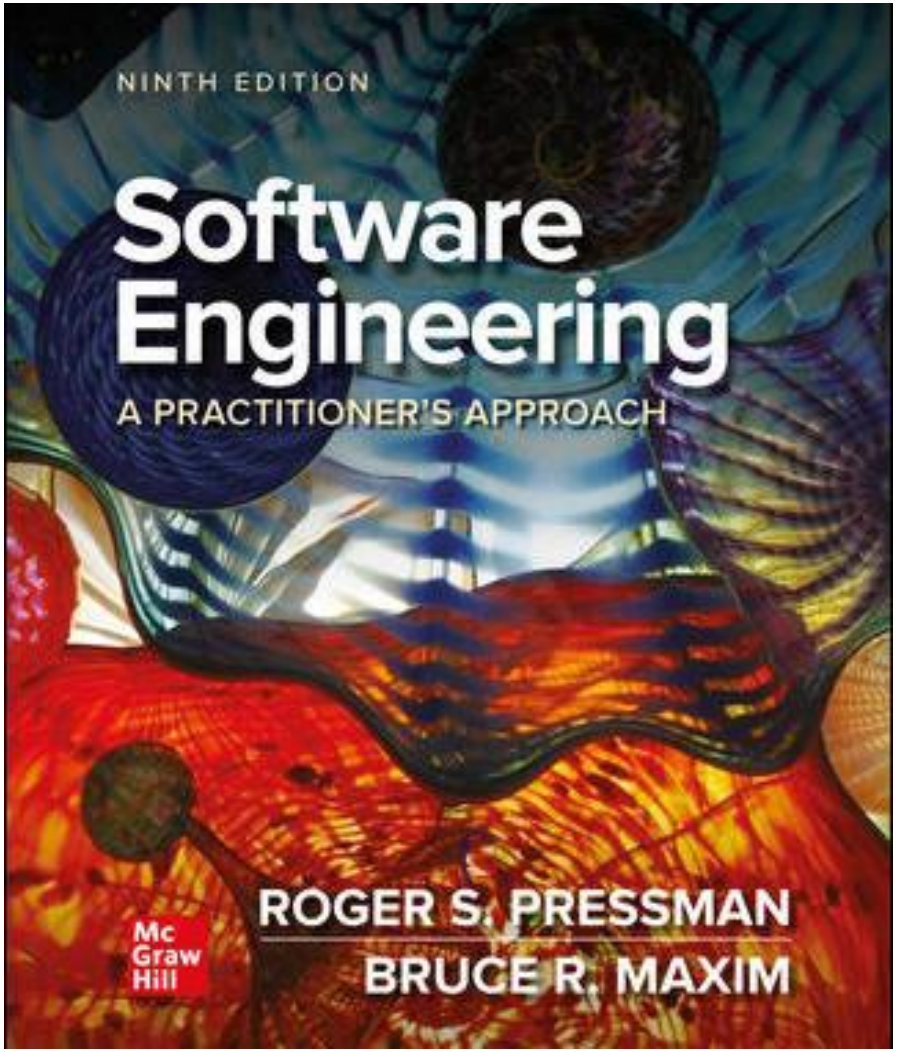
Mc Graw Hill Education

Because learning changes everything.®



Chapter 29

Emerging Trends in Software Engineering

Part Five – Advanced Topics

Technology Evolution

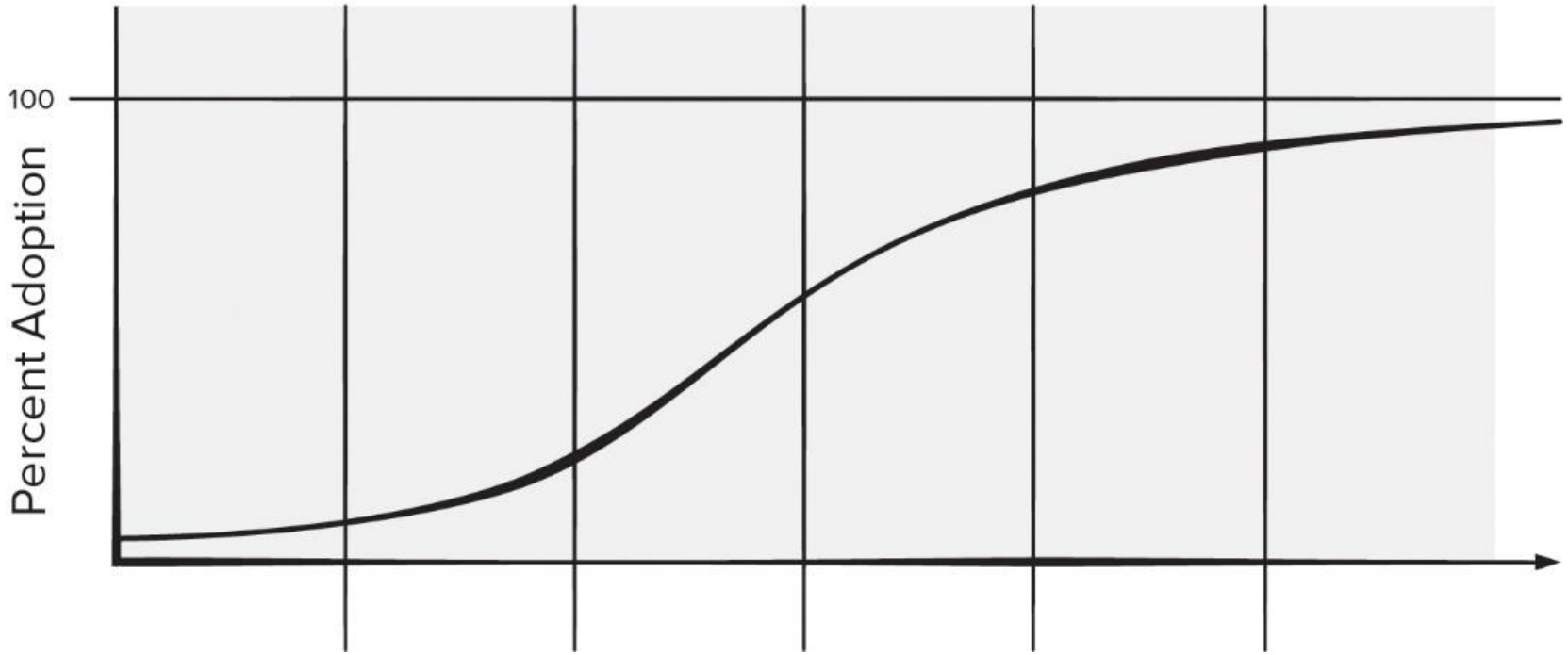
Challenges faced when trying to isolate technology trends:

- ***What Factors Determine the Success of a Trend*?**
- ***What Lifecycle Does a Trend Follow*?**
- ***How Early Can a Successful Trend be Identified*?**
- ***What Aspects of Evolution are Controllable*?**

Ray Kurzweil argues that technological evolution is similar to biological evolution but occurs at a rate that is orders of magnitude faster.

- Evolution (whether biological or technological) occurs as a result of positive feedback—"the more capable methods resulting from one stage of evolutionary progress are used to create the next stage."

Technology Innovation Cycle



Access the text alternative for slide images.

Innovation Life Cycle

- ***Breakthrough phase*** - problem is recognized and repeated attempts at a viable solution are attempted.
- ***Replicator phase*** - initial breakthrough work is reproduced in the and gains wider usage.
- ***Empiricism phase*** - leads to the creation of empirical rules that govern the use of the technology, and
- ***Theory phase*** - repeated success leads to a broader theory of usage.
- ***Automation phase*** - creation of automated tools.
- ***Maturity phase*** - technology matures and used widely.

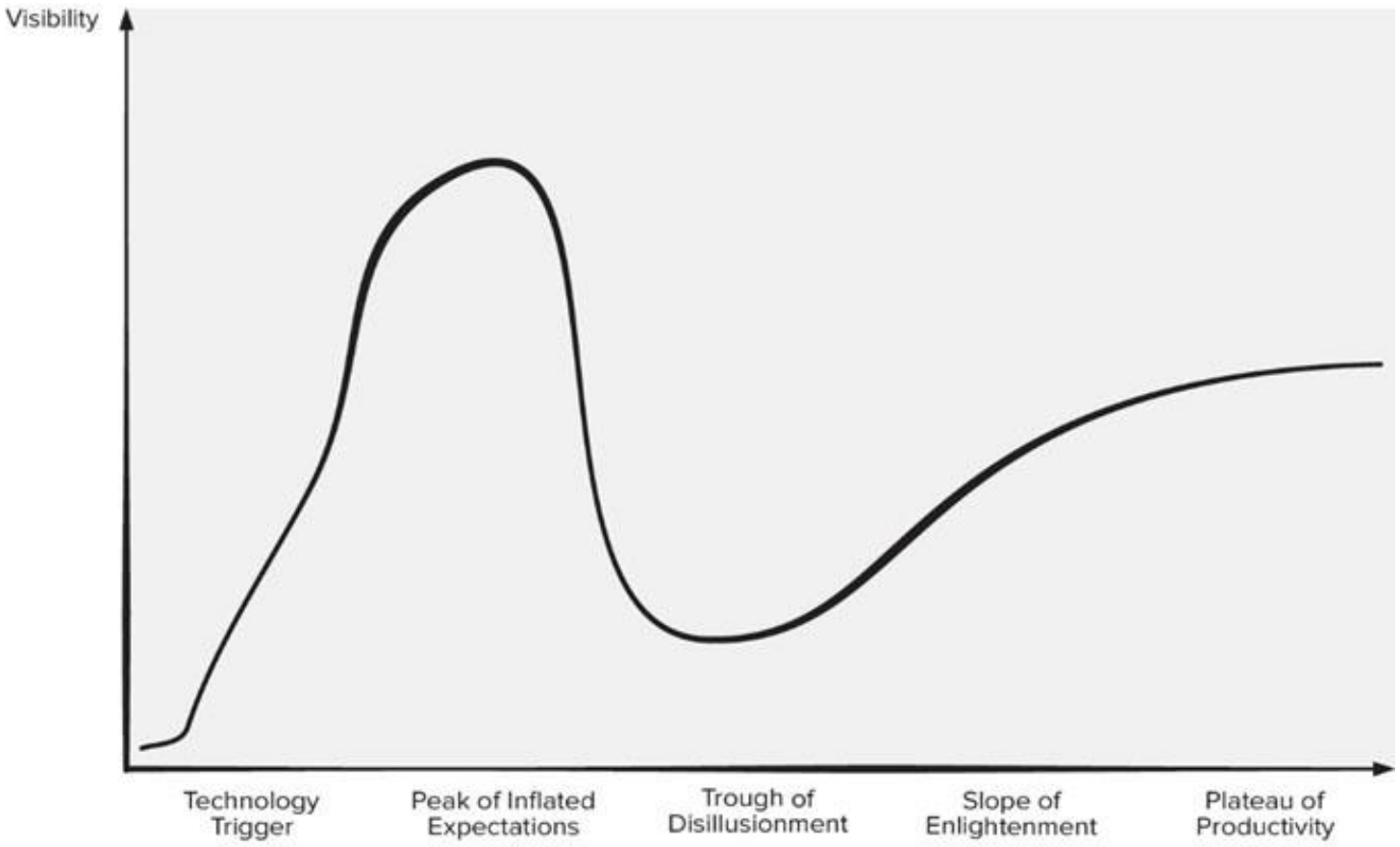
Observing Software Engineering Trends

- Barry Boehm suggests that:

  software engineers face the formidable challenges of dealing with rapid change, uncertainty and emergence, dependability, diversity, and interdependence, but they also have opportunities to make significant contributions that will make a difference for the better.

- But what of more modest, short-term innovations, tools, and methods?
- Gartner Group - has developed a *hype cycle for emerging technologies,*

Hype Cycle



Access the text alternative for slide images.

Source: Linden, Alexander, Fenn, "Understanding Gartner's Hype Cycles," Strategic Analysis Report. Inc.., May 30 2003, 5.

Identifying Soft Trends

- ***Connectivity and collaboration*** has already led to a software teams that do not occupy the same physical space.
- ***Globalization*** - leads to a diverse workforce.
- ***An aging population*** - many experienced software engineers and managers will be leaving the field over the coming decade and their knowledge must be captured.
- ***Consumer spending in emerging economies*** will double and a non-trivial percentage of this spending will be applied to products and services that have a digital component requiring software.

Managing Complexity

Systems requiring over 1 billion LOC will begin to emerge:

- Consider the interfaces for a billion LOC system (outside world, interoperable systems, Internet, internal components).
- Is there a reliable way to ensure that these connections will allow information to flow properly?
- Consider the project itself.
- Consider the number of people (and their locations) who will be doing the work.
- Consider the engineering challenge.
- Consider the challenge of quality assurance.

Open-World Software

- **Open-world software** - software that is designed to adapt to a continually changing environment ‘by self-organizing its structure and self-adapting its behavior.”
- Concepts such as *ambient intelligence, context-aware applications,* and *pervasive/ubiquitous computing* - all focus on integrating software-based systems into an environment far broader than anything to date.
- Engineering of *variability intensive systems* focuses on systems that can be highly variable during all software engineering activities and we need to improve our understanding of how to design and manage them in a cost-effective manner.

Emergent Requirements

As systems become more complex requirements will emerge as everyone involved in the engineering and construction of a complex system learns more about the systems, the environment in which it resides, and the users who will interact with it.

This implies several of software engineering trends:

- Process models must be designed to embrace change and adopt the basic tenets of the agile philosophy.
- Methods that yield engineering models (for example, requirements and design models) must be used judiciously because those models will change repeatedly as more knowledge about the system is acquired.
- Tools that support both process and methods must make adaptation and change easy.

Software Building Blocks

- Software engineering community attempts to capture past knowledge and reuse proven solutions, but a significant percentage of the software that is built today continues to be built “from scratch.”
- Part of the reason for this is a desire by stakeholders and software engineering practitioners for “unique solutions.”
- There a move to use components packaged as merchant software and a tendency to adopt *software platform solutions* to incorporate collections of related functionalities provided within an integrated software framework.

Open Source

- ***Open source -*** *development method for software that harnesses the power of distributed peer review and transparency of process.*
- *The promise of open source is better quality, higher reliability, more flexibility, lower cost, and an end to predatory vendor lock-in.*
- The term ***open source*** when applied to computer software, implies that software engineering work products (models, source code, test suites) are open to the public and can be reviewed and extended (with controls) by anyone with interest and permission.

Process Trends

1. *As SPI frameworks evolve, they will emphasize "strategies that focus on goal orientation and product innovation."*
2. *Because software engineers have a good sense of where the process is weak, process changes should generally be driven by their needs and should start form the bottom up.*
3. *Automated software process technology (SPT) will move away from global process management to focus on aspects of software process that can best benefit from automation.*
4. *Greater emphasis will be placed on the return-on-investment of SPI activities.*
5. *As time passes, the software community may come to understand that expertise in sociology and anthropology may have as much of more to do with successful SPI as other, more technical disciplines.*
6. *New modes of learning may facilitate the transition to a more effective software process.*

Grand Challenge

- Software-based systems will undoubtedly become bigger and more complex.
- Software engineers can meet the challenge of complex software development by can be managed only if the software engineering community develops more effective collaborative software engineering philosophy, better requirements engineering approaches, robust approach to model-driven development, and better software tools.
- Techniques used for variability intensive systems (continuous delivery, self-adaptive software, value-based software engineering, content aware computing) may help developers of all software products.

Collaborative Development

- Software engineers collaborate across time zones and international boundaries, and they must share information.
- The same holds for open-source projects in which hundreds of software developers work to build product.
- Crowd sourcing has been suggested as a means of enhancing coverage test cases generated by automated testing tools.
- Information must be disseminated so that coordination and collaboration can occur.

Requirements Engineering

To improve the manner in which requirements are defined, the software engineering community will likely implement three distinct sub-processes as RE:

- Improved knowledge acquisition and knowledge sharing that allows more complete understanding of application domain constraints and stakeholder needs.
- Greater emphasis on iteration as requirements are defined.
- More effective communication and coordination tools that enable all stakeholders to collaborate effectively.

Model-Driven Software Development

- ***Model-drive software development*** - couples domain-specific modeling languages with transformation engines and generators in a way that facilitates the representation of abstraction at high levels and then transforms it into lower levels.
- ***Domain-specific modeling languages*** (DSMLs) represent application structure, behavior, and requirements within a particular application domains are described with meta-models.
- These DSML meta-models are tuned to design concepts inherent in the application domain and can therefore represent relationships and constraints among design elements in an efficient manner.

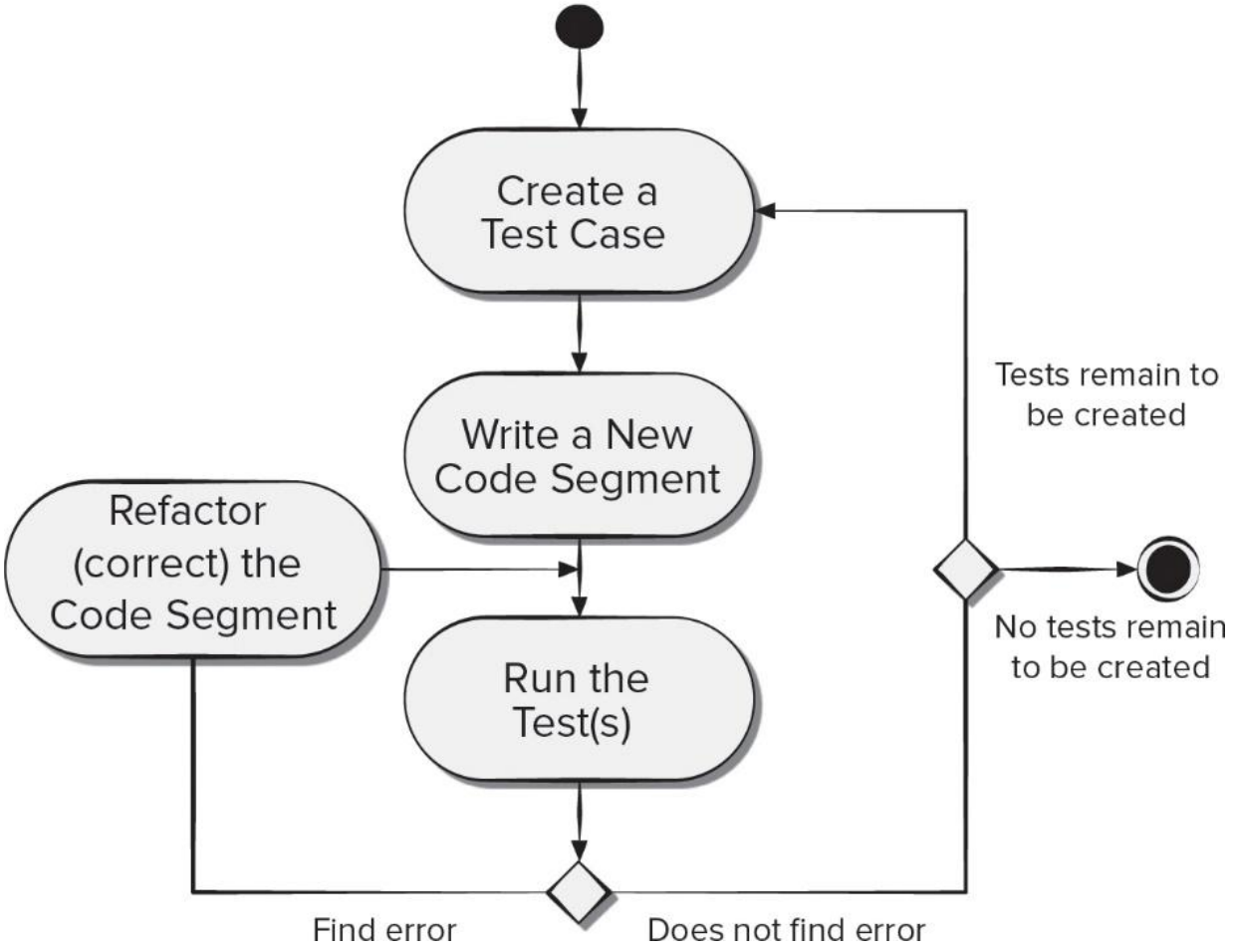
Search-Based Software Engineering

- ***Search-based software engineering*** (SBSE) applies metaheuristic search techniques such as genetic algorithms to software engineering problems.
- SBSE is based on the premise that it is often easier to check that a candidate solution solves a problem than it is to construct a solution from scratch.
- Genetic improvement of software has resulted in performance improvements in existing software products.
- Search-based software engineering techniques have been used to generate and repair sequences of refactoring recommendations.

Test-Driven Development

- ***Test-driven development*** (TDD) - requirements for a software component serve as the basis for the creation of a series of test cases that exercise the interface and try to find errors in data structures or functionality delivered.
- The code is designed and written to satisfy the test.
- If the passes, a new test is created for the next segment of code to be developed.
- The process continues until the component is fully coded and all tests execute without error.
- If any test succeeds in finding an error, the existing code is refactored, and all tests created to that point are executed again.

Test-Driven Development Process Flow



Access the text alternative for slide images.

Tool-Related Trends

- One dominant trend in technology tools is the creation of a tools that support model-driven development with an emphasis on architecture-driven design.
- Software engineers are beginning to build tools that focus on the interaction of humans and technology.
- New tools that support stakeholder collaboration will emerge as important as tools support for technology.
- Complete software engineering environments, point-solution tools that address everything from requirements gathering to design/code refactoring to testing will continue to evolve and become more functionally capable.

Because learning changes everything.®

www.mheducation.com